

MAREA ROUTE

Estrategias de navegacion en la arquitectura de software

Actividad 6 - Navegando mareas | Septiembre 2026

Equipo	Integrantes (3-4)
Roles	Lider / Diseñador UML / Programador / Investigador
Docente	(nombre del docente)

Rutas suboptimas cuando el trafico cambia

- **Problema:** los mapas calculan la ruta mas corta/rapida sobre un plan estatico de la red.
- **Marea de trafico:** en horas pico la congestion satura calles centrales con apariencia de atajo.
- **Comun:** la ruta de menor distancia no es la mas rapida en minutos reales.
- **Pregunta:** como arquitecturar un navegador para que la estrategia sea intercambiable, reaccione a la congestion y presente el recorrido de forma extensible?

Impacto en tres dimensiones

- **Experiencia de usuario:** menos tiempo de viaje, menos stres, instrucciones claras paso a paso.
- **Rendimiento del sistema:** A* reduce nodos explorados y la latencia del servicio de mapas.
- **Eficiencia operativa:** flotas y entregas ahorran combustible y horas-hombre con replanificacion en vivo.
- **Escalabilidad:** nuevas estrategias de ruteo se agregan sin tocar el nucleo (interfaz RouteStrategy).

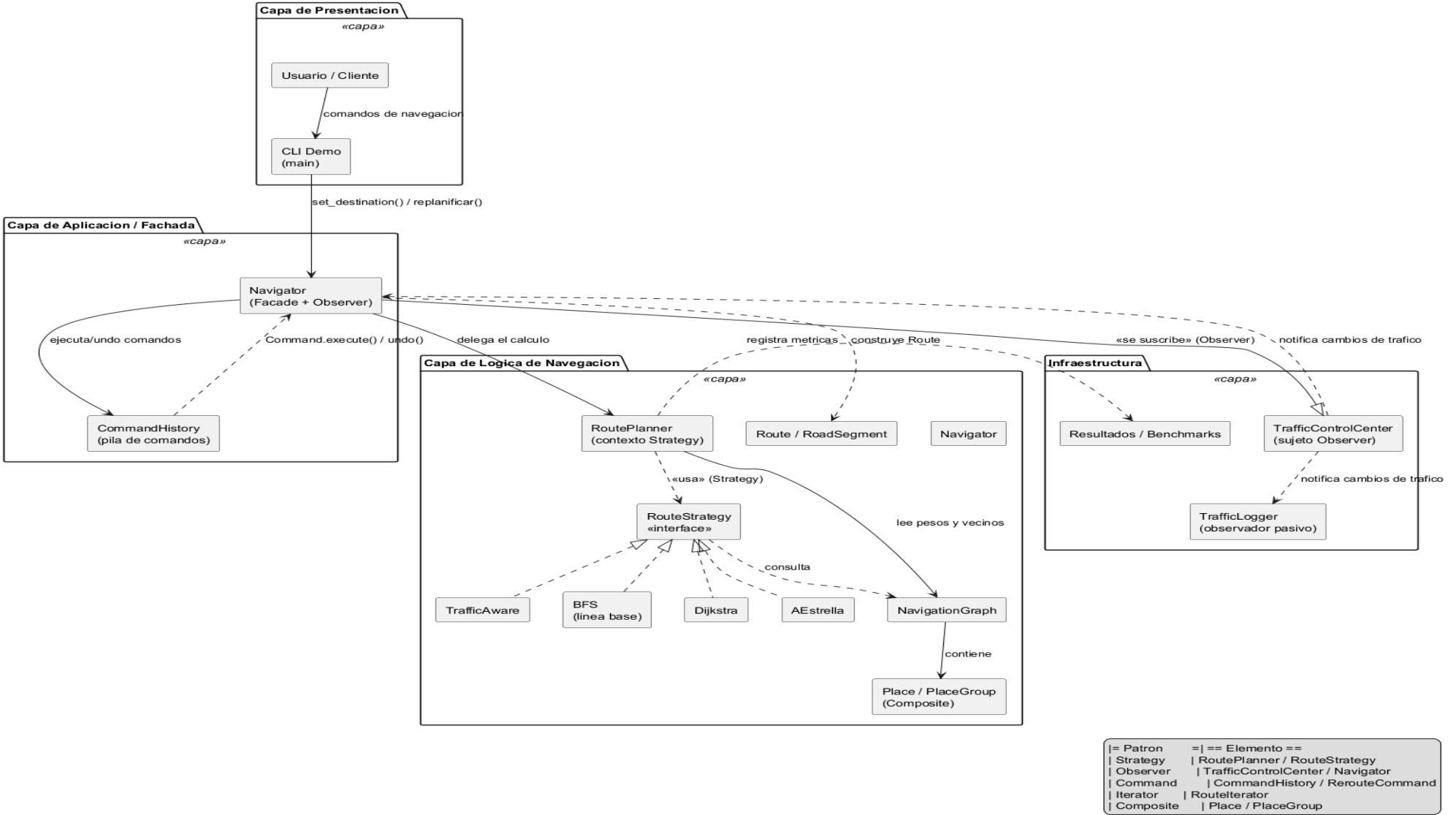
Capas con fachada central y patrones desacoplados

Capa	Rol	Patrones
Presentacion (CLI)	Demo por consola	-
Aplicacion	Navigator (fachada) + CommandHistory	Facade, Observer, Command
Logica de ruteo	RoutePlanner, RouteStrategy, grafo y rutas	Strategy, Composite, Iterator
Infraestructura	TrafficControlCenter, metricas	Observer

Regla: las dependencias apuntan a interfaces y los patrones se modelan antes en UML.

Diagrama de componentes

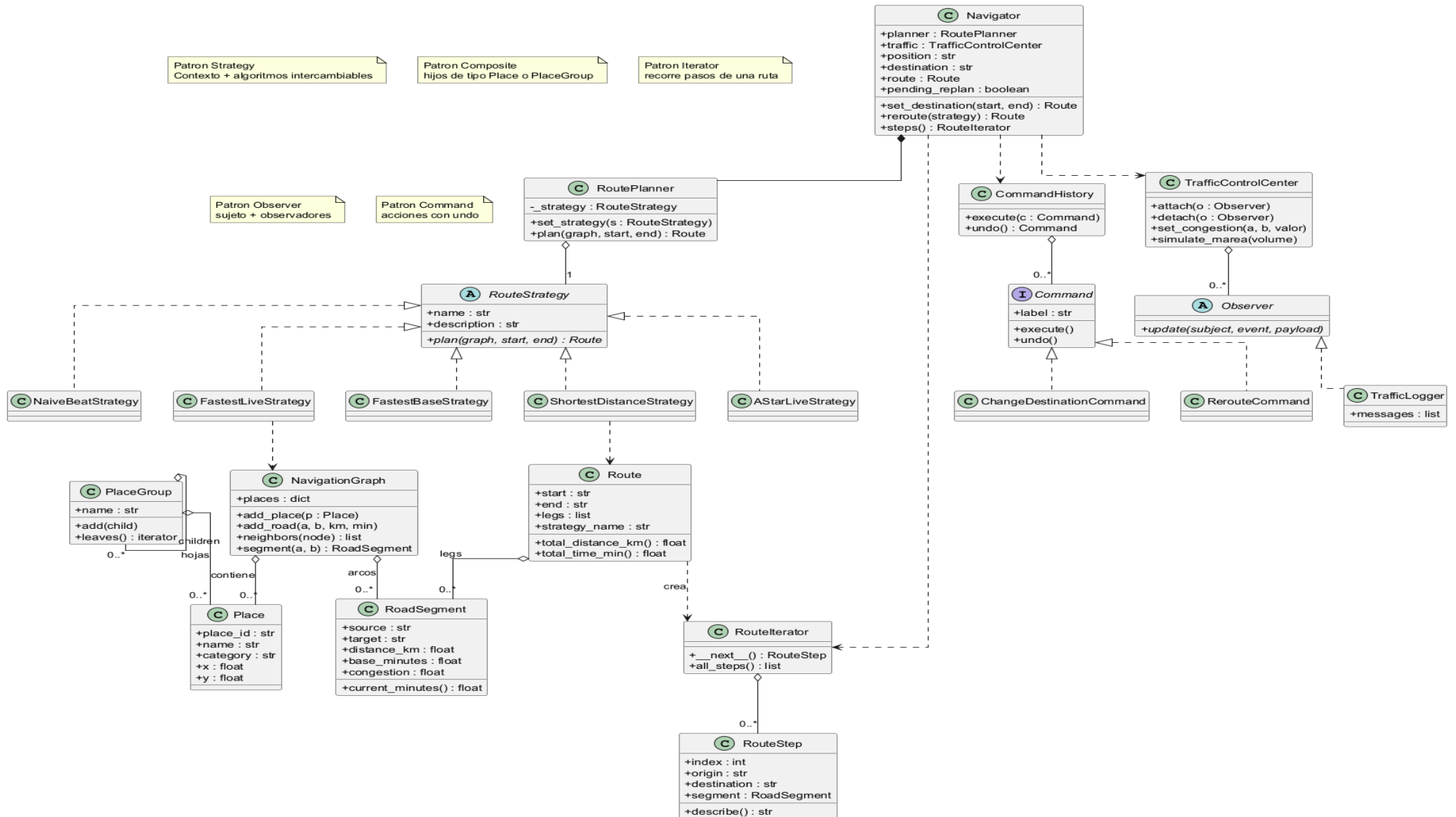
Diagrama de Componentes - MAREA ROUTE



Estructura y conectores entre capas.

Diagrama de clases

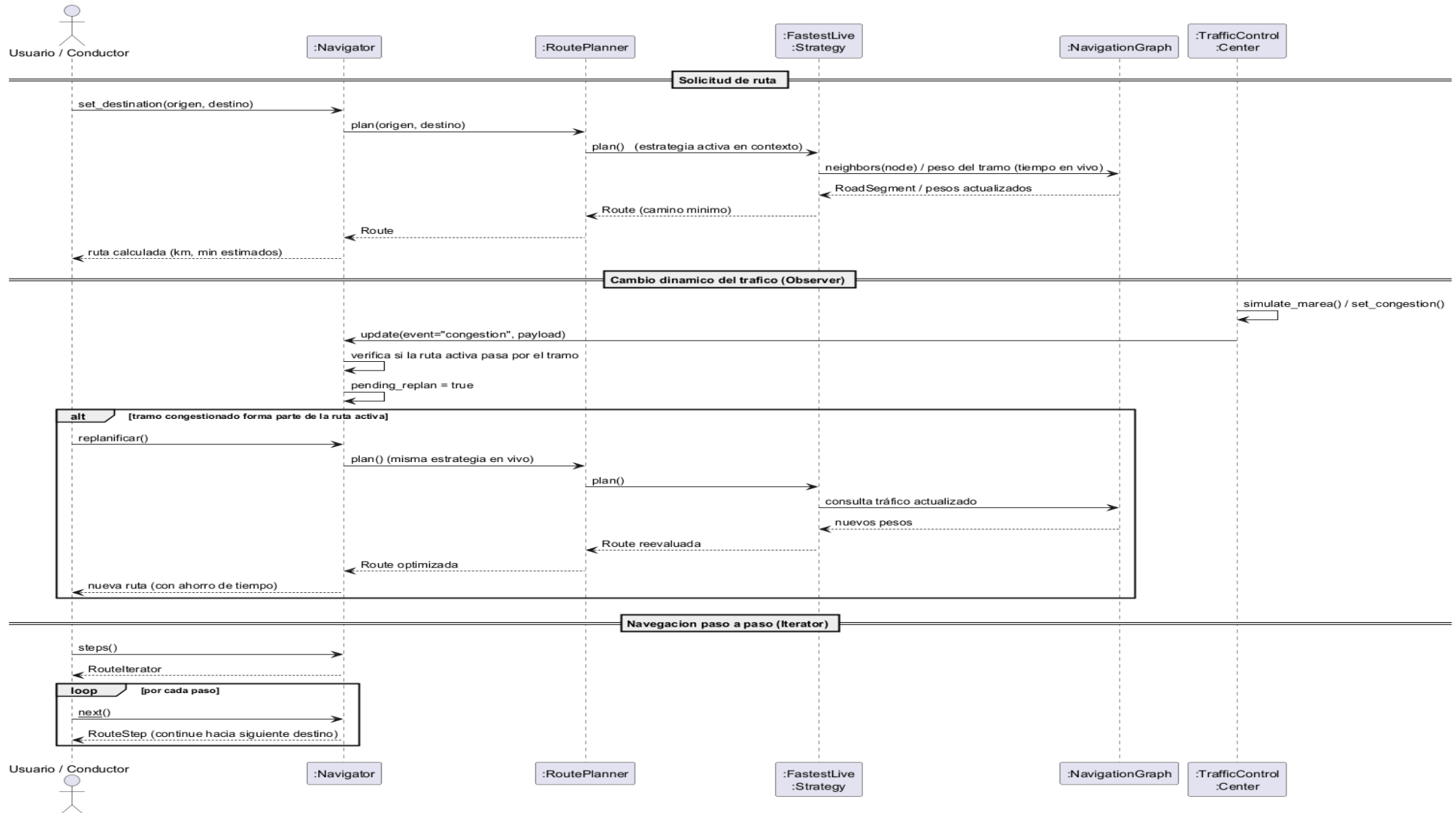
Diagrama de Clases - MAREA ROUTE



Colaboracion de los cinco patrones.

Diagrama de secuencia

Diagrama de Secuencia - Navegacion con replanificacion



Solicitud de ruta, notificacion y replanificacion.

Cinco patrones para navegar optimamente

Patron	Aplicacion en MAREA ROUTE
Strategy	RoutePlanner conmuta Dijkstra, A* y modo en vivo (mareas).
Observer	TrafficControlCenter notifica a Navigator; activa replanificación.
Command	RerouteCommand / ChangeDestinationCommand con undo en CommandHistory.
Iterator	RouteIterator entrega instrucciones paso a paso desacopladas.
Composite	PlaceGroup navega grupos de destinos como un único destino.

Python 3 estandar, reproducible y probado

[4] PATRON STRATEGY: comparacion de estrategias UNAB -> Estadio

+-----+-----+-----+-----+-----+						
Estrategia	km	min plano	min marea	nodos	ms	
+-----+-----+-----+-----+-----+						
BFS - saltos (linea base)	15.46	16.87	17.52	11	0.03	
Dijkstra - minima distancia	10.03	21.87	29.74	15	0.03	
Dijkstra - tiempo plano (sin trafico)	11.25	13.65	19.78	11	0.02	
Dijkstra - tiempo en vivo (marea)	15.46	16.87	17.52	11	0.02	
A* - tiempo en vivo + heuristica	15.46	16.87	17.52	10	0.03	
+-----+-----+-----+-----+-----+						

- **11 pruebas unitarias:** optimo de Dijkstra, A* == Dijkstra en vivo, Observer, Command, Iterator, Composite.
- **Reproduccion:** python -m navigation.demo | python -m unittest discover -s tests | python -m benchmarks.run_benchmarks

Metricas: 30 pares OD x 5 estrategias x 2 escenarios

Estrategia	km	min (marea)	nodos	ms
BFS - saltos (linea base)	12.85	19.53	73.5	0.093
Dijkstra - minima distancia	11.15	18.07	67.8	0.156
Dijkstra - tiempo plano (sin trafico)	11.15	18.26	70.8	0.165
Dijkstra - tiempo en vivo (marea)	11.45	16.94	71.5	0.185
A* - tiempo en vivo + heuristica	11.45	16.94	48.5	0.229

- **Ahorro en vivo vs minima distancia (marea): 1.13 min** promedio por viaje, con recorridos ligeramente mayores.
- **A* expande 32.2% menos nodos** para el mismo camino optimo que Dijkstra.
- **Ruta en vivo supera al plan estatico en 7.2%** durante la marea.

CONCLUSIONES

Optimizar no es solo acortar distancias

- La arquitectura por capas + fachada aislaron algoritmos, trafico y presentacion.
- Strategy hizo medible la optimizacion: el modo en vivo gana durante la marea.
- Observer permitio replanificar sin intervencion del usuario.
- Command, Iterator y Composite sumaron usabilidad y extensiones sin tocar el nucleo.
- Trabajo futuro: grafos jerarquicos, dato historico/en vivo y API REST de rutas.

Video de demostracion: enlace privado (YouTube) incluido en el informe y el LMS.

Reproduccion y evidencias

```
python -m navigation.demo
```

```
python -m unittest discover -s tests
```

```
python -m benchmarks.run_benchmarks
```

Evidencias: diagramas UML en docs/uml/, metricas en docs/results/benchmark.json, informe en docs/informe/.

Gracias.